TASK 1 — WEB APPLICATION QA & DEBUG REPORT

Automation & QA Developer | Take-Home Skills Assessment

Prepared by	Syed Anees	**Application under test**	RealWorld demo (demo.realworld.show)
Assessment area	Registration, input validation, email identity, API authentication	**Report date**	10 November 2026

1. Executive Summary

This report documents five issues observed during exploratory testing of the RealWorld web application. The checks focused on account registration, username validation, password policy, email identity handling, and authentication enforcement on protected API endpoints. The findings below are based on the behavior observed during the assessment and should be reproducible using the steps provided.

Evidence note: Screenshots or a screen recording should be attached to the final submission where available. The authentication finding is classified as Critical in this report based on the reported ability to access protected API functionality without supplying the returned token. Confirm the exact request headers, cookies, endpoint, and returned data in the accompanying evidence before treating the impact as fully established.

2. Findings Summary

#	Finding	Area	Severity
1	Duplicate username registration is accepted	Registration	**Medium**
2	Special characters are accepted in usernames	Input validation	**Low**
3	Weak passwords are accepted during registration	Password policy	**Medium**
4	Gmail dot variations can register separate accounts	Email identity	**High**
5	Protected API access reportedly succeeds without the returned token	API authentication	**Critical**

3. Detailed Bug Reports

Finding 1. Duplicate username registration is accepted

Category	Registration / identity validation	**Severity**	**Medium**
Steps to reproduce	1. Register an account using a username (for example, **syan03**). 2. Start a separate registration using the same username and a different email address. 3. Submit the second registration and observe whether it is accepted.	**Expected result**	The application should enforce username uniqueness and reject a username that is already registered.
Actual result	The application reportedly accepts registration using a username that has already been used.	**Suspected cause**	The registration flow may not perform a reliable uniqueness check, or the database may lack an effective unique constraint on the username.
Potential impact	Duplicate usernames can create identity ambiguity and may cause incorrect profile attribution or authorization issues if other parts of the application assume usernames are unique.		

Finding 2. Special characters are accepted in usernames

Category	Input validation	**Severity**	**Low**
Steps to reproduce	1. Open the registration form. 2. Enter a username containing characters such as **>**, **"**, and **<** (for example, **>"<**). 3. Submit the form and observe whether registration accepts the value.	**Expected result**	The application should enforce a documented username character policy and safely handle accepted input wherever it is displayed.
Actual result	The application reportedly accepts the supplied special characters in the username.	**Suspected cause**	Username validation may be too permissive or missing. Output encoding must also be checked wherever the username is rendered.
Potential impact	Acceptance alone does not prove XSS. Risk depends on whether the value is safely encoded when displayed or used in other contexts.		

Finding 3. Weak passwords are accepted during registration

Category	Registration / password policy	**Severity**	**Medium**
Steps to reproduce	1. Open the registration form. 2. Enter otherwise valid registration details. 3. Set a weak password (the tester observed that a weak password was accepted). 4. Submit the form and confirm whether the account is created.	**Expected result**	The application should enforce an appropriate password policy and communicate the requirements to users.
Actual result	The application reportedly allows registration with a weak password.	**Suspected cause**	Password-strength requirements may be absent or insufficiently enforced by the server-side registration logic.
Potential impact	Weak passwords can increase susceptibility to password guessing and credential-stuffing attacks, particularly when users reuse passwords.		

Finding 4. Gmail dot variations allow multiple registrations

Category	Email identity / registration	**Severity**	**High**
Steps to reproduce	1. Register an account using the original Gmail address (for example, **syedanees843@gmail.com**). 2. Attempt another registration using a dotted variation of the same Gmail mailbox. 3. Complete the email verification process for the second registration using the same mailbox. 4. Observe whether the application creates separate accounts.	**Expected result**	The application should apply a consistent email identity policy before checking uniqueness and creating an account. It should prevent duplicate identities where the addresses resolve to the same mailbox under that policy.
Actual result	The tester reports that the original Gmail address and its dotted variation were accepted as separate registrations and that both verification messages reached the same mailbox.	**Suspected cause**	The application may compare raw email strings without applying the intended normalization policy, and may not enforce uniqueness on the normalized identity.
Potential impact	Multiple application accounts associated with one mailbox can cause identity confusion and may affect account recovery, verification, or abuse controls. The exact impact depends on the application's email identity and recovery design.		

Finding 5. Protected API access reportedly succeeds without the returned authentication token

Category	API authentication / access control	**Severity**	**Critical**
Steps to reproduce	1. Submit valid credentials to the login endpoint using a test account. 2. Observe the successful response containing account details and an authentication token. 3. Remove the token from the login response in the test flow. 4. Send a request to a protected API endpoint without an **Authorization** header and without relying on an existing authenticated browser session or session cookie. 5. Observe whether the endpoint returns protected data or permits the protected operation.	**Expected result**	Protected API endpoints should independently enforce authentication and reject requests that do not contain a valid authentication credential. An unauthenticated request should receive an appropriate denial response, typically HTTP 401.
Actual result	The tester reports that protected API endpoints remained accessible after the token was removed from the login response. The precise endpoint, response body, status code, and request headers should be included in the attached evidence.	**Suspected cause**	The API may have missing or inconsistent authentication middleware, may trust an unintended session mechanism, or may fail to validate credentials on one or more protected routes.
Potential impact	If confirmed on a fresh unauthenticated client, this could permit unauthorized access to protected account data or functionality. Severity should be reassessed against the exact data and actions exposed.		

4. Root-Cause Analysis — Authentication Enforcement

During testing, a login request using valid test credentials returned HTTP 200 together with account details and an authentication token. The token was then removed from the login response, after which the tester reported being able to access protected API functionality. This behavior suggests that the protected endpoint may not be consistently enforcing authentication, or that an alternate session mechanism is being accepted. A likely area to investigate is the authentication middleware and route-level authorization configuration. Each protected endpoint should validate an approved credential on every request rather than relying only on the fact that a login request previously succeeded. The fix should ensure that requests without a valid token or other explicitly supported session credential are rejected. Regression tests should cover missing, malformed, expired, and invalid credentials across all protected routes. The finding should be validated from a fresh client with no cookies or cached session state to rule out an existing browser session.

5. Testing Notes and Evidence

The findings in this report reflect the tester's observations during the assessment. The report does not claim that the application's source code was reviewed or that the suspected causes were confirmed through code analysis. Attach a screen recording or screenshots showing the relevant registration attempts, the same-mailbox verification behavior, and the unauthenticated API request and response. For the authentication finding, preserve the request method, endpoint, response status, relevant headers, and a redacted response sample. Do not include real passwords, live authentication tokens, verification codes, or other sensitive personal data in the evidence.

6. Recommended Remediation Summary

Finding	Recommended action
Duplicate username	Validate username uniqueness on the server and enforce it with a database unique constraint.
Special characters	Define and enforce an explicit username format; contextually encode all output and test rendering paths.
Weak passwords	Implement a clear password policy and enforce it server-side; consider breached-password screening and rate limiting.
Gmail variations	Define supported email normalization rules, check uniqueness on the normalized identity, and enforce the rule atomically in the database.

Finding	Recommended action
Authentication bypass	Require valid authentication on every protected route; reject missing/invalid credentials and add regression tests for all protected endpoints.

End of report. Severity ratings are based on the behavior described by the tester. Final severity may change after confirming reproducibility, scope, and demonstrated impact.